Launch Readiness — The First 1,000 Clients

50pick · `50pick.tz` · prepared 3 September 2026. Every number in this document was measured against live production on the day of writing — from the Railway API, the production database and the live site itself. Nothing here is copied from internal documentation, because two of the figures that documentation carries turned out to be out of date.

50pick works. It is fast, it has not dropped a single request in the sample we took, its money handling is genuinely well defended, and last night's backup restores. What it has never had is **a crowd**. Production currently serves **two to five betting customers a day**, and the busiest market in its history took 29 bets from 12 people. Opening to 1,000 clients is a step up of roughly two hundred times.

This document says what we run today, what it costs today, what has to change before the doors open, and what it will cost after. It ends with the honest list of what could still go wrong.

> **The single most important sentence in this document:** as of today **no member of the public can register on 50pick**, because registration requires an SMS code and the platform is configured to deliver SMS to a log file rather than to a telephone. That is a signature on a contract, not a piece of engineering — but until it is done, nothing else in this plan matters.

1 · What we run today

```
Players (Tanzania)
   |   no CDN, no firewall, nothing cached at the edge
   v
Railway edge network  ---->  ORIGIN: us-west2  (California)
   |
   +-- 50pick        Next.js 15    1 container   no health check, no drain
   +-- PostgreSQL 18.6              1 container   5 GB disk (897 MB used)
   +-- Redis                        1 container   connected and in use

Outside Railway:
   Selcom ......... deposits and withdrawals          LIVE
   Postmark ........ email                            LIVE, 20 sent, 0 failures
   Cloudflare R2 ... customer ID documents + backups   LIVE
   Sentry .......... error reporting                   LIVE
   Anthropic ....... market generation + auto-settling  LIVE
   Twelve Data ..... price feed for Up & Down          LIVE
   SMS ............. one-time codes                    NOT DELIVERING
```

31 ms	0	104	17 h
typical server response	server errors in 500 requests	registered customers	since backup last verified

The account is on Railway's **Pro** plan. That matters, because it means the ceilings people worry about are not close: disks can grow to 1 TB on demand without downtime, and up to 42 copies of the application may run at once. We are nowhere near either.

> **Two things the internal notes had wrong, both in our favour.** The code and documentation repeatedly state that the database allows only 100 simultaneous connections and that Redis is switched off in production. Measured live: the database allows **500**, and Redis is **connected and working across containers**. Several "we cannot scale" conclusions written earlier in the year were based on those two stale figures and no longer hold.

2 · What it costs today

Taken from Railway's billing API for the workspace *Ali Sheib's Projects*, and from the platform's own AI spend ledger.

Railway — the last four bills

Billing period	Invoiced
May → June	$5.00
June → July	$11.15
July → August *(last complete month)*	$25.42
20 Aug → today *(14 days in, still running)*	$17.38

Railway — where this month's money is going

Project	Processor	Memory	Disk	Traffic	Total
50pick	$1.50	**$20.60**	$0.10	$1.33	**$23.57**
generous-elegance	$0.02	$3.89	$0.15	$0.01	$4.07
amanat-watan	$0.20	$2.12	$0.11	$0.40	$2.83
intercontinental-glass	—	$0.92	$0.09	—	$1.01
lmc-sagesse	—	$0.33	—	—	$0.33
All six projects	**$1.72**	**$27.86**	**$0.45**	**$1.74**	**$31.81**

50pick is about three-quarters of the Railway bill, and **memory alone is 87% of 50pick's share**. The application sits almost idle on processing power while holding an average of 4.7 GB of memory. That is the whole Railway cost story in one line, and it is the cheapest saving available to us.

Artificial intelligence — the largest single line

What it does	Calls (30 days)	Average each	30-day cost
Writing new markets for customers to bet on	185	$0.185	$34.15
Checking and auto-settling finished markets	72	$0.230	$16.59
Customer help chat	5	$0.002	$0.01

Total	**262**		**$50.75**

> **The AI bill is larger than the entire hosting bill**, and this is the most useful thing we learned about cost. Crucially, **99.98% of it does not depend on how many customers we have** — it is the cost of generating and settling markets, which happens on a clock whether one person is watching or ten thousand. Going from 104 customers to 1,000 will barely move it. The only part that grows with customers is the help chat, which has cost one cent in a month.

Everything together, today

Service	Per month	Basis
Railway — 50pick	$23.57	measured
Railway — five other projects	$8.24	measured
Anthropic (AI)	$50.75	measured
Postmark (email)	$0	20 emails sent — inside free allowance
Cloudflare R2 (documents, backups)	~$1	under 1 GB stored
Sentry (error reporting)	$0	free tier
Twelve Data (price feed)	not measured	billed outside Railway — to confirm
SMS	$0	not sending anything
Running total	**≈ $84 / month**	

3 · What has to happen before we open

Ranked by what stops us, not by how hard it is. The first two are decisions and configuration. The rest is engineering.

🔴 1 · Turn on SMS `BLOCKS EVERYTHING`

Signing up requires a one-time code sent by text message. The platform is set to the `console` provider, which writes *"NOT delivered"* into a log and throws the code away. Production has never successfully sent a single one. **No new customer can join today.**

The Selcom connection is already written and waiting — it needs three settings: the provider name, an API key, and a sender ID licensed by the TCRA. **This is a commercial step, not a technical one.** Once it is in, we prove it by registering on a real handset and reading the code off the screen — not by looking at a log.

🔴 2 · Put the admin console behind two-factor

Second-factor authentication is switched off for all nine administrator accounts, on a licensed real-money platform. Internal notes say it was meant to be turned on at the end of July. We enrol an authenticator app and prove a full sign-in with a real code *first*, then remove the override — doing it the other way round risks locking everyone out of the platform.

🔴 3 · Stop every update being an outage

All three services run as a single copy, and Railway has **no health check configured**, no overlap between the old and new version during an update, and no grace period for requests already in progress. Today that means every deployment briefly takes the site down and cancels any bet being placed at that moment. With five customers nobody notices. With a thousand, everybody does.

There is a subtlety worth stating plainly: the platform's own health page currently reports *"healthy"* even when the database is unreachable. Pointing Railway's health check at it as it stands would create a safety check that cannot fail, which is worse than none. We fix the health page first, then switch the check on, then **deliberately break the database to watch it go red**.

🟠 4 · Make big markets able to pay out

When a market finishes, winners are paid one after another inside a single database operation with a thirty-second limit. Measured, that runs out at roughly **660 winners**. The largest market we have ever run had 29 bets, so this has never been reached — but a popular football match with a thousand customers will pass it comfortably, and beyond that point the market freezes for both betting and cashing out while it repeatedly times out. We break the payout into batches and prove it by settling a test market with over a thousand winners.

🟠 5 · Stop rebuilding every page from scratch

Nothing on 50pick is cached. Every page is rebuilt from the database on every view, and the busiest screens automatically refresh themselves every 15 to 20 seconds. The Up & Down board costs around forty database queries each time it is drawn. At 200 customers online that is roughly four hundred database queries a second purely to redraw screens whose contents have barely changed.

Redis is already installed, already connected, and currently used for almost nothing. Caching those screens for a handful of seconds is invisible to customers — they refresh every twenty seconds anyway — and removes the large majority of that load. **This is the highest-value change in the whole plan.**

🟠 6 · Move the servers closer to Tanzania

The application and database both run in California. The server itself answers in 31 milliseconds, but a customer waits around half a second, because the request has to cross the planet and back. That delay is geography, not slow software — and it is the same on every page, which is exactly what distance looks like.

Moving to Amsterdam roughly halves it, and **costs nothing extra**. The database has to move with the application, and moving stored data requires a short planned outage, so we do it in an announced window using the platform's existing maintenance mode — which pauses betting and deposits while **leaving withdrawals open**, so no customer's money is ever locked in.

🟡 7 · Give the database room

The database disk is 897 MB of a 5 GB allowance and grows a measured **14 MB a day**. That is about six months of headroom. Notably, **89% of that growth is the platform's own automatic market activity, not customers**, so a thousand customers will not change it much.

It still needs raising, because the compliance record inside it can never be deleted — it is required to be kept for seven years and is cryptographically chained, so removing old rows would break it. Growing the disk takes seconds, needs no downtime, and **costs nothing until the space is actually used**, because Railway charges for what is stored rather than what is reserved.

4 · What it will cost with 1,000 clients

Sized for **1,000 registered customers with 100–200 online at once** at peak — the figure agreed for this launch. We are deliberately *not* running multiple copies of the application: one well-tuned container handles this comfortably once caching is in place, and running several would first require fixing a set of emergency controls that currently only reach one copy.

Line	Today	At 1,000	Why it changes
Railway — memory	$20.60	$25 – 55	lower end if we cap memory; higher if left as-is
Railway — processor	$1.50	$10 – 18	caching holds this down
Railway — database	$5.30	$8 – 12	more queries, more storage
Railway — Redis	~$0	$2 – 4	now doing real caching work
Railway — disk & backups	$0.14	$1 – 3	charged on what is used
Railway — traffic	$1.33	$10 – 15	roughly ten times the visitors
Railway — other five projects	$8.24	$8 – 10	unchanged
Anthropic (AI)	$50.75	$55 – 110	market generation is flat; only help chat grows
SMS one-time codes	$0	$35 – 45	new — roughly 4,000 messages a month
Postmark (email)	$0	$0 – 15	free up to 100 messages a month
Cloudflare R2	~$1	$1 – 3	more ID documents stored
Total	**≈ $84**	**$155 – 290**	

> **Call it $200 a month, and watch two lines.** The AI bill and the memory bill are the only two that can run away, and both are directly controllable — the first by how often we generate markets, the second by capping how much memory the application is allowed to hold. Twelve Data's price-feed plan is billed outside Railway and still needs confirming; it is the one figure in this document I have not been able to measure.

5 · Safety

What is already built, tested, and should not be touched

This platform has been through real incidents and the scars are visible in the code, in a good way. All of the following are working and have automated tests proving them:

- **Maintenance mode** pauses new bets and deposits while leaving withdrawals open, so customer money is never trapped by an operational decision.
- **Payment kill switches** per mobile network, separately for money in and money out, for use during a provider incident.

- **Payments are settled from a signed re-query, not from the incoming message**, exactly once, and defended against a tampered amount. A deposit is recovered by a 15-second poller and a 5-minute sweep *even if the payment provider's confirmation never arrives*.
- **A dead Redis cannot break a bet** — proven by a test that points the platform at a dead port and asserts that betting still succeeds.
- **Repeated bets cannot double-charge.** A retry without an idempotency key is refused outright, so a retry can never become a second bet.
- **Money invariants hold under load**: 200 simultaneous bets on one market, zero shillings lost or created.
- **The Up & Down safety net is deliberately independent of the game's own off switch** — turning the game off must never turn off the thing that returns money already staked in it.
- **Backups are verified by actually restoring them** into a scratch database every night and re-checking the money totals — not merely by confirming a file was written.

What is still open

Gap	Severity	Fixed in
No customer can register — SMS delivers nothing	🔴 Critical	Step 1
Admin console has no second factor (9 accounts)	🔴 Critical	Step 2
Health page reports healthy while the database is down	🔴 Critical	Step 3
Up to 24 hours of data could be lost — no continuous backup	🔴 Critical	Step 8
Nobody is alerted when a backup fails	🔴 Critical	Step 9
No written recovery procedure or on-call plan	🟠 High	Step 10
Recovery time never measured at current data size	🟠 High	Step 10
Updates cancel bets in progress	🟠 High	Step 3
If the database is down at start-up, the app will not start at all	🟠 High	Step 3
Backup key and document-store credentials readable from one place	🟡 Medium	after launch
No firewall or denial-of-service protection in front of the site	🟡 Medium	after launch

> **The August backup failure is the story worth remembering.** The nightly backup failed on **eleven consecutive nights**. The most recent restorable copy became nearly twelve days old, on a platform holding real customer money. Nothing was broken about the alarm — the compliance screen displayed the warning correctly for ten of those days. **Nobody was looking at it.** The remedy that was shipped is a command an operator has to remember to run, which means the same failure would happen the same way again. Step 9 replaces it with an alert that arrives on its own, and we will test it by breaking the backup on purpose.

6 · If it crashes, how do we get back?

Honestly stated: **we would recover, but more slowly than we should, and we have never timed it.** The pieces exist and are good; they have simply never been assembled into a procedure that someone could follow at two in the morning.

Question	Answer today	After this plan
How much data could we lose?	Up to **24 hours** — one backup a night, nothing in between	**Minutes**, via continuous point-in-time recovery
How long to get back up?	**Unknown** — rehearsed once in July against a database a tenth of today's size	Measured and written down
Is there a written procedure?	**No**	A one-page recovery runbook
Would we know it had happened?	Error reporting yes; backup failure **no**	Alerts that arrive without being asked
Can we trade while degraded?	**Yes** — maintenance mode already works	Unchanged, and part of the procedure

Railway also provides its own point-in-time recovery for the database, which is separate from our nightly backup. Turning it on gives us **two unrelated ways to recover**, so a mistake in one does not cost us the other. That is step 8, and it is the single largest reduction in risk available for the least work.

7 · The order of work

#	Step	How we prove it is done
1	Turn on Selcom SMS	Register on a real handset; read the code off the phone
2	Enrol and enforce admin two-factor	Full admin sign-in with a real code, before removing the override
3	Health check, deploy overlap, graceful shutdown	Break the database on purpose and watch the check go red
4	Grow the database disk	New size visible; site stays up throughout
5	Cache the busy screens in Redis	Re-measure page timing and database query rate, before and after
6	Move to Amsterdam	Re-run the timing test against today's half-second baseline
7	Batch the payout of big markets	Settle a test market with over 1,000 winners and time it
8	Turn on continuous database recovery	Perform a real point-in-time restore into a scratch database
9	Alerts that arrive by themselves	Break the backup deliberately; confirm the alert lands
10	Rehearse recovery; write the runbook	Record the actual wall-clock recovery time

> **Steps 1 and 2 are decisions; the rest is a few days of engineering.** Nothing in this list is a rewrite, and nothing requires new infrastructure to be bought. The platform is closer to ready than the length of this document suggests — the gaps are almost entirely in operations and recovery, not in the product or in how it handles money.

Prepared 3 September 2026 against production commit `78c3876e`. All figures measured live: Railway billing API, Railway metrics API, the production PostgreSQL database, and timed requests to `50pick.tz`. The one figure not measured is the Twelve Data subscription, which is billed outside Railway. Rebuild this document with `node docs/runbooks/mkpdf.mjs ../launch-1k.html 50pick-launch-1k.pdf`.